TP 02 — Simuler la propagation d'une épidémie

Inside ESILV : une journée dans la peau d'un élève-ingénieur

Sixtine Michel

De Vinci Higher Education — ESILV

ESILV
DE VINCI ENGINEERING SCHOOL

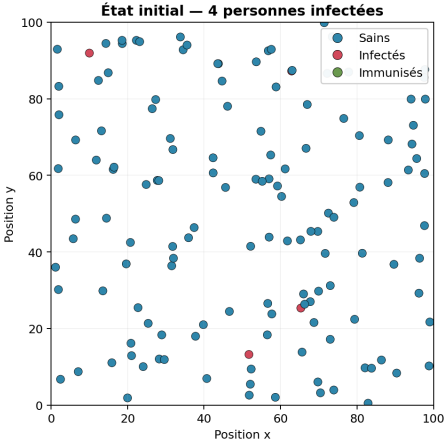
Notre mission

Question

Comment des règles individuelles très simples peuvent-elles produire une épidémie à l'échelle d'une population ?

Chaque point représente une personne qui possède :

- une position (x, y) ;
- une vitesse (v_x, v_y) ;
- un état sanitaire.



D'un point de vue algorithmique

Une génération de l'épidémie enchaîne quatre opérations :

```
def creer_population():
    # Initialiser positions, vitesses et etats.
def deplacer_individus(population):
    # Avancer et rebondir sur les bords.
def contaminer_population(population):
    # Tester les contacts avec les infectes.
def mettre_a_jour_infections(population):
    # Faire passer les gueris a l'etat immunise.
```

La visualisation répète ensuite ces opérations à chaque image.

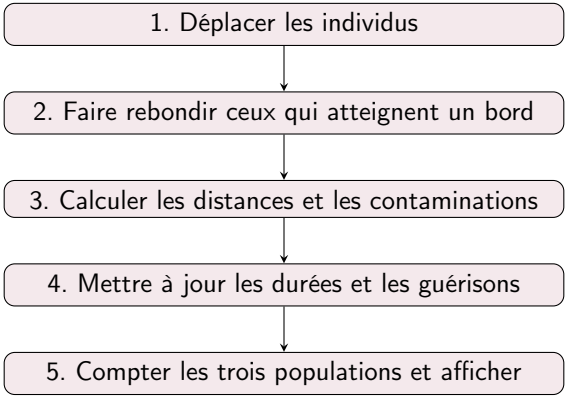
Trois états possibles



Simplification du modèle

Une personne immunisée ne redevient pas malade. Le modèle décrit une idée algorithmique ; il ne cherche pas à prévoir une véritable épidémie.

Une génération de la simulation



Correspondance entre les étapes et les TODO

Avant de lancer la boucle, les **TODO 1 à 3** créent la population initiale : positions, vitesses et premiers individus infectés.

1. **Déplacer les individus**	**TODO 4**
2. **Faire rebondir sur les bords**	**TODO 5 et 6**
3. **Calculer les distances et les contaminations**	**TODO 7 à 10**
4. **Mettre à jour les durées et les guérisons**	**TODO 11 et 12**
5. **Compter les trois populations**	**TODO 13**

Affichage

Le rafraîchissement des graphiques est déjà fourni : il utilise les positions, les états et les compteurs calculés pendant ces cinq étapes.

Créer la population — TODO 1 à 3

Les tableaux NumPy contiennent une ligne par personne et deux colonnes pour les coordonnées x et y.

Créer la population — TODO 1 à 3

Les tableaux NumPy contiennent une ligne par personne et deux colonnes pour les coordonnées x et y.

```
# TODO 1 : positions dans le carre [0, TAILLE_MONDE]
positions = np.random.uniform(
    0, TAILLE_MONDE, size=(NB_INDIVIDUS, 2)
)

# TODO 2 : composantes de vitesse
vitesses = np.random.uniform(
    -VITESSE_MAX, VITESSE_MAX,
    size=(NB_INDIVIDUS, 2)
)

# TODO 3 : indices distincts, sans remise
infectes = np.random.choice(
    NB_INDIVIDUS, NB_INFECTES_INITIAUX, replace=False
)
```

Déplacer les individus — TODO 4

Pour chaque personne i, le nouveau point est obtenu par une addition de vecteurs :

$$x_i^{n+1} = x_i^n + v_i.$$

Déplacer les individus — TODO 4

Pour chaque personne i, le nouveau point est obtenu par une addition de vecteurs :

$$x_i^{n+1} = x_i^n + v_i.$$

```
# positions et vitesses ont la meme forme : (NB_INDIVIDUS, 2)
positions[:] = positions + 1.0 * vitesses
```

Intérêt de NumPy

Une seule instruction réalise ici les 140 additions vectorielles.

Rebondir à gauche et à droite — TODO 5

On examine la coordonnée x et la vitesse horizontale v_x.

Rebondir à gauche et à droite — TODO 5

On examine la coordonnée x et la vitesse horizontale v_x.

```
for i in range(NB_INDIVIDUS):
    if positions[i, 0] < 0:       # gauche
        positions[i, 0] = 0
        vitesses[i, 0] = -vitesses[i, 0]
    if positions[i, 0] > TAILLE_MONDE:  # droite
        positions[i, 0] = TAILLE_MONDE
        vitesses[i, 0] = -vitesses[i, 0]
```

L'indice 0 désigne l'axe horizontal. Inverser v_x fait repartir la personne vers l'intérieur du monde.

Rebondir en bas et en haut — TODO 6

On applique le même raisonnement à y et à la vitesse verticale v_y.

Rebondir en bas et en haut — TODO 6

On applique le même raisonnement à y et à la vitesse verticale v_y.

```
for i in range(NB_INDIVIDUS):
    if positions[i, 1] < 0:       # bas
        positions[i, 1] = 0
        vitesses[i, 1] = -vitesses[i, 1]
    if positions[i, 1] > TAILLE_MONDE:  # haut
        positions[i, 1] = TAILLE_MONDE
        vitesses[i, 1] = -vitesses[i, 1]
```

L'indice 1 désigne l'axe vertical. Les quatre conditions sont maintenant indépendantes et lisibles directement.

Chercher un contact — TODO 7 et 8

On compare chaque personne saine aux personnes infectées.

Chercher un contact — TODO 7 et 8

On compare chaque personne saine aux personnes infectées.

```
for i in range(NB_INDIVIDUS):
    if etats[i] == SAIN:              # TODO 7
        infecte_proche = False

        for j in range(NB_INDIVIDUS):
            if etats[j] == INFECTE:   # TODO 8
                # calculer la distance entre i et j
```

Les indices `i` et `j` désignent deux personnes différentes à comparer. Le booléen mémorise si un contact dangereux a été trouvé.

Calculer la distance — TODO 9

Pour les positions (x_i, y_i) et (x_j, y_j) :

$$d = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}.$$

Calculer la distance — TODO 9

Pour les positions (x_i, y_i) et (x_j, y_j) :

$$d = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}.$$

```
dx = positions[j, 0] - positions[i, 0]
dy = positions[j, 1] - positions[i, 1]
distance = np.sqrt(dx ** 2 + dy ** 2)

if distance < RAYON_CONTAMINATION:
    infecte_proche = True
    break
```

`break` arrête la recherche dès qu'un infecté proche est trouvé.

Décider une contamination — TODO 10

Deux conditions sont nécessaires : un contact proche et un tirage aléatoire favorable.

Décider une contamination — TODO 10

Deux conditions sont nécessaires : un contact proche et un tirage aléatoire favorable.

```
if (
    infecte_proche
    and np.random.random() < PROBABILITE_CONTAMINATION
):
    nouveaux_infectes.append(i)
```

Pourquoi attendre la fin de la boucle ?

Les nouveaux états sont appliqués ensemble, afin d'éviter une réaction en chaîne artificielle au cours d'une seule génération.

Guérir et devenir immunisé — TODO 11 et 12

À vous

- Si i est infecte, augmenter sa duree d'infection de 1.
- Immuniser i lorsque la duree maximale est atteinte.

Guérir et devenir immunisé — TODO 11 et 12

```
for i in range(NB_INDIVIDUS):
    if etats[i] == INFECTE:          # TODO 11
        durees[i] = durees[i] + 1

        if durees[i] >= DUREE_INFECTION:  # TODO 12
            etats[i] = IMMUNISE
```

Une personne saine ou déjà immunisée ne passe pas dans le premier `if`.

Compter la population — TODO 13

À vous

- Parcourir les etats et incrementer le bon compteur.

Compter la population — TODO 13

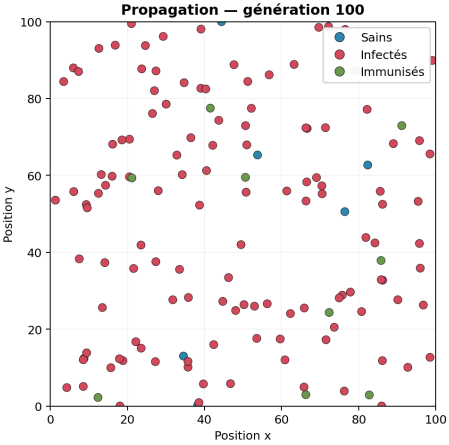
```python
sains = infectes = immunises = 0

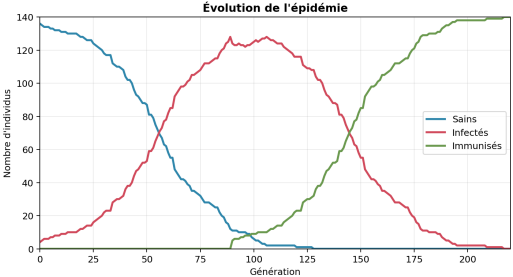
for etat in etats:
    if etat == SAIN:
        sains = sains + 1
    elif etat == INFECTE:
        infectes = infectes + 1
    elif etat == IMMUNISE:
        immunises = immunises + 1

return sains, infectes, immunises
```

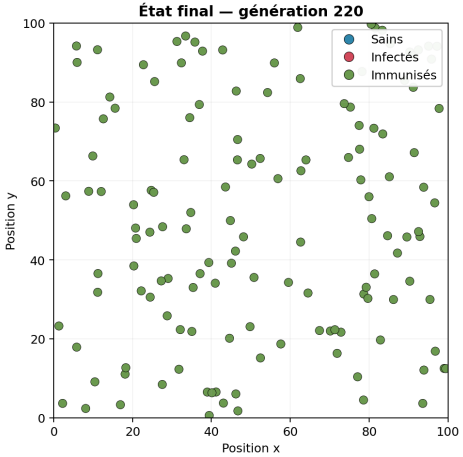
Vérification : $N_{\text{sains}} + N_{\text{infectes}} + N_{\text{immunises}} = N_{\text{individus}}$.

Observer la propagation





État final



À retenir

Des règles locales simples font apparaître une dynamique globale : croissance, pic, puis disparition de l'infection.

Le résultat varie d'une exécution à l'autre car les déplacements et les contaminations sont aléatoires.

Expérimenter

Relancer la simulation après avoir modifié un seul paramètre :

- doubler `RAYON_CONTAMINATION` ;
- diviser `PROBABILITE_CONTAMINATION` par deux ;
- raccourcir `DUREE_INFECTION` ;
- ralentir les déplacements avec `VITESSE_MAX`.

Question

Quel paramètre modifie le plus la hauteur et la date du pic d'infection ?

Lancer le TP dans Spyder

1. Ouvrir `02-propagation_epidemie.py`.
2. Choisir un backend graphique **Automatic** ou **Qt**.
3. Redémarrer la console IPython si le backend vient d'être modifié.
4. Compléter les TODO dans l'ordre et exécuter le fichier après chaque groupe d'étapes.

Affichage dynamique

L'instruction `plt.show(block=True)` maintient la fenêtre ouverte pendant l'animation.

Concepts appris

- tableaux NumPy ;
- positions et vecteurs ;
- masques booléens ;
- distance euclidienne ;
- conditions combinées ;
- probabilités ;
- évolution d'un état ;
- visualisation dynamique.

À vous de coder !